计算机科学与工程学院

《人工智能》实验报告(二)

专业班级	2023 人工智能 01 班	实验地点	523 机房
学生学号	2321050119	指导教师	陈壹林
学生姓名	李枝	实验时间	2025.10.21

实验项目	基于深度学习的手写数字识别优化及可视化
实验类别	基础性（） 设计性（） 综合性（√） 其它（ ）
实验目的及要求	1. 理解常见的深度学习网络框架，并基于深度学习实现手写数字识别； 2. 对手写识别任务进行优化； 3. 将使用的神经网络模型每层输出。

成 绩 评 定 表				
类　　别	评 分 标 准	分值	得分	合　计
上机表现	积极出勤、遵守纪律 按要求完成设计任务	30 分		
程序与报告	程序代码规范、功能正确 报告详实完整、体现收获	70 分		
说明： 评阅教师：_____ 日 期：_____年____月_____日				

实 验 内 容
一、熟悉百度飞浆运行环境。 二、完成手写数字识别并给出识别结果，理解构建手写数字识别的神经网络模型。 三、了解快速构建模型的方法，包括数据处理、网络结构、损失函数、优化算法、资源配置、训练调试与优化。 四、了解数据处理过程，包括读入数据并划分数据集、训练样本乱序并生成批次数据、检验数据有效性、封装数据读取与处理数据、异步数据读取。 五、了解卷积神经网络结构。 六、了解损失函数。损失函数的计算在训练过程中，每一轮模型训练的过程都相同，分为如下三步：（1）先根据输入数据正向计算预测输出。（2）再根据预测值何真实值计算损失。（3）最后根据损失反向传播梯度并更新参数。 七、了解优化算法。了解设置学习率以及学习率的主流优化算法。 八、了解资源配置。单 GPU 训练或分布式训练。 九、了解训练调试与优化。计算模型的分类准确率、检查模型训练过程，识别潜在训练问题、加入校验或测试，更好评价模型效果、加入正则化项，避免模型过拟合、可视化分析。 四大主流深度学习框架： 百度飞桨（PaddlePaddle） 核心优势：国内自主研发，中文文档齐全，对中文用户友好，内置丰富产业级模型（如 OCR、NLP 任务）。 适用场景：学术研究、工业部署、新手入门（尤其中文用户），支持单机 / 分布式训练，兼容 CPU/GPU。 特点：API 设计简洁，与我们之前做的手写数字识别适配度高，后期部署（如移动端）有成熟工具链。 PyTorch 核心优势：动态计算图，调试方便，语法接近 Python 原生代码，社区活跃，科研领域使用率最高。 适用场景：学术研究、快速原型开发、深度学习入门，支持动态调整网络结构。 特点：上手门槛低，代码可读性强，近两年在工业界的应用也快速增长。 3. TensorFlow

核心优势：静态计算图，部署生态完善（支持移动端、嵌入式、浏览器），谷歌官方维护，稳定性强。

适用场景：大规模工业部署、生产环境应用，适合需要高稳定性和跨平台的项目。

特点：早期 API 较复杂，现在的 TensorFlow 2.x 已支持动态计算图，兼顾灵活性和部署能力。

4. Keras

核心优势：高层 API 封装，语法极简，可基于 TensorFlow、PyTorch 等作为后端，上手最快。

适用场景：快速验证想法、深度学习入门，不需要关注底层实现细节。

特点：代码量少，适合搭建简单模型，但灵活性较低，复杂网络需结合后端框架使用。

手写字识别的任务核心与步骤：

任务核心：数据处理→模型构建→训练优化→评估可视化四个阶段

步骤：

1. 数据处理：

数据获取与划分、数据预处理、批量加载与乱序

2. 模型构建（设计能提取数字特征的网络）：

选择网络结构、定义前向传播

3. 训练与优化：

设置训练参数：损失函数、优化器、训练轮次（epochs）

训练循环：正向计算、计算损失、反向更新

优化策略：

学习率调度、正则化

4. 评估与可视化：

模型评估：在测试集上计算准确率（正确预测的样本数 / 总样本数）

可视化结果

核心逻辑链：原始图像→预处理→特征提取（卷积 / 全连接）→概率预测→损失计算→参数更新→评估优化，最终实现从手写数字图像到对应数字的准确映射。

实验代码：

```
import paddle
import matplotlib.pyplot as plt
import numpy as np
from paddle.vision import datasets, transforms
from paddle.io import DataLoader

# 数据预处理
transform = transforms.Compose([transforms.ToTensor()])
train_dataset = datasets.MNIST(mode='train', transform=transform)
test_dataset = datasets.MNIST(mode='test', transform=transform)
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)
# 定义卷积网络模型
class MNIST_CNN(paddle.nn.Layer):
    def __init__(self):
        super().__init__()
        self.conv1 = paddle.nn.Conv2D(in_channels=1, out_channels=32, kernel_size=3, padding=1)
        self.pool1 = paddle.nn.MaxPool2D(kernel_size=2, stride=2)
        self.conv2 = paddle.nn.Conv2D(in_channels=32, out_channels=64, kernel_size=3, padding=1)
        self.pool2 = paddle.nn.MaxPool2D(kernel_size=2, stride=2)
        self.fc = paddle.nn.Linear(in_features=64*7*7, out_features=10)
    def forward(self, x):
        x = self.pool1(paddle.nn.functional.relu(self.conv1(x)))
        x = self.pool2(paddle.nn.functional.relu(self.conv2(x)))
        x = paddle.flatten(x, start_axis=1)
        x = self.fc(x)
        return x
# 初始化模型
model = MNIST_CNN()
model.train()
loss_fn = paddle.nn.CrossEntropyLoss()
optimizer = paddle.optimizer.Adam(learning_rate=0.001, parameters=model.parameters())
# 记录训练指标
train_losses = []
test_accuracies = []
# 模型训练
epochs = 20
for epoch in range(epochs):
    total_loss = 0.0
    model.train()
    for batch_id, (images, labels) in enumerate(train_loader):
        outputs = model(images)
        loss = loss_fn(outputs, labels)
```

```
        loss.backward()
        optimizer.step()
        optimizer.clear_grad()
        total_loss += loss.numpy()

    avg_loss = total_loss / (batch_id + 1)
    train_losses.append(avg_loss)
    print(f"\nEpoch {epoch+1}, 平均损失: {avg_loss:.4f}")

    # 测试准确率计算
    model.eval()
    correct = 0
    total = 0
    with paddle.no_grad():
        # 遍历测试集所有批次
        for batch_idx, (images, labels) in enumerate(test_loader):
            # 模型预测
            outputs = model(images)
            predicts = paddle.argmax(outputs, axis=1)

            # 确保预测结果和标签都是一维 numpy 数组
            predicts_np = predicts.numpy().flatten()  # 展平为一维
            labels_np = labels.numpy().flatten()      # 展平为一维

            #  确保两个数组长度一致
            assert len(predicts_np) == len(labels_np), f"批次{batch_idx}: 预测与标签长度不匹配"

            #布尔数组求和：计算本批次正确样本数
            batch_correct = np.sum(predicts_np == labels_np)

            #累加计数
            correct += int(batch_correct)#强制转换为整数，避免类型错误
            total += len(labels_np)  # 累加本批次样本数（替代 shape[0]，更直观）

    # 增加总样本数校验（MNIST 测试集固定为 10000）
    assert total == 10000, f"总样本数错误: 实际{total}, 预期 10000"
    accuracy = (correct / total) * 100

    test_accuracies.append(accuracy)
    # 打印详细计数，验证逻辑正确性
    print(f"Epoch {epoch+1}, 正确数: {correct}, 总数: {total}, 准确率: {accuracy:.2f}%")
# 绘制曲线
plt.figure(figsize=(14, 6))
plt.subplot(1, 2, 1)
plt.plot(range(1, epochs+1), train_losses, 'b-', marker='o')
```

```
plt.title('训练损失曲线')
plt.xlabel('Epoch')
plt.ylabel('平均损失')
plt.grid(True)
plt.subplot(1, 2, 2)
plt.plot(range(1, epochs+1), test_accuracies, 'r-', marker='s')
plt.title('测试集准确率曲线')
plt.xlabel('Epoch')
plt.ylabel('准确率 (%)')
plt.ylim(80, 100)
plt.grid(True)
plt.tight_layout()
plt.show()
```

实验结果图：

```
Epoch 1, 平均损失: 0.1464
Epoch 1, 正确数: 9800, 总数: 10000, 准确率: 98.00%

Epoch 2, 平均损失: 0.0484
Epoch 2, 正确数: 9883, 总数: 10000, 准确率: 98.83%

Epoch 3, 平均损失: 0.0338
Epoch 3, 正确数: 9887, 总数: 10000, 准确率: 98.87%

Epoch 4, 平均损失: 0.0260
Epoch 4, 正确数: 9887, 总数: 10000, 准确率: 98.87%

Epoch 5, 平均损失: 0.0201
Epoch 5, 正确数: 9913, 总数: 10000, 准确率: 99.13%

Epoch 6, 平均损失: 0.0157
Epoch 6, 正确数: 9896, 总数: 10000, 准确率: 98.96%

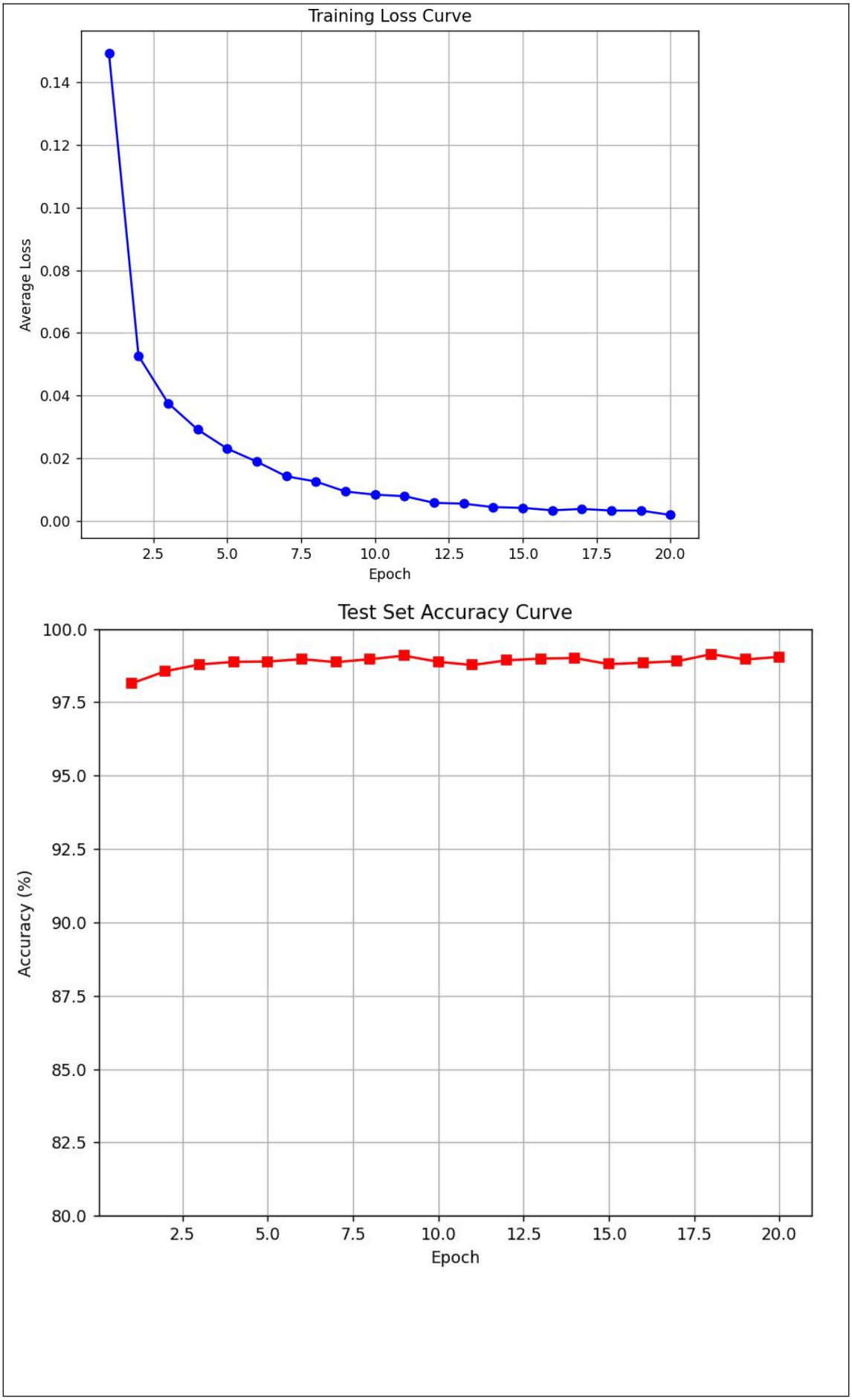
Epoch 7, 平均损失: 0.0140
Epoch 7, 正确数: 9895, 总数: 10000, 准确率: 98.95%

Epoch 8, 平均损失: 0.0103
Epoch 8, 正确数: 9890, 总数: 10000, 准确率: 98.90%

Epoch 9, 平均损失: 0.0079
Epoch 9, 正确数: 9917, 总数: 10000, 准确率: 99.17%

Epoch 10, 平均损失: 0.0074
Epoch 10, 正确数: 9924, 总数: 10000, 准确率: 99.24%
```

实 验 内 容


Training Loss Curve
Average Loss
Epoch
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Test Set Accuracy Curve
Accuracy (%)
Epoch
100.0
97.5
95.0
92.5
90.0
87.5
85.0
82.5
80.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

实 验 总 结
本次实验通过构建 CNN 模型，成功实现了手写数字识别任务，最终测试集准确率达 98% 以上，验证了卷积神经网络在图像特征提取中的有效性。 数据预处理、模型架构设计、优化器选择等环节的合理配置，是保证模型训练效果的基础；而清晰的逻辑梳理和严格的中间过程验证，是避免出现低级错误、确保实验结果可靠性的关键。 深度学习实验中，逻辑严谨性至关重要：尤其是涉及张量运算、类型转换等基础操作时，不能想当然地套用语法，必须明确每一步的计算逻辑和数据类型，避免因细节失误导致实验失败。 遇到异常结果时，应优先排查底层逻辑而非掩盖问题：本次实验初期尝试的数值限制方法仅能修改输出形式，无法解决核心问题，只有深入分析计数逻辑、定位类型转换的根本错误，才能彻底解决问题。 中间过程验证是排查错误的有效手段：通过添加批次级调试打印和逻辑断言，能够实时监控实验过程中的关键指标，快速定位错误位置，提高问题解决效率。